

INSURIFIT-Texas Health Insurance Chatbot

CS6374 Semester Project Report

Code Repository-https://github.com/SurajJayakumar/INSURIFIT_chatbot

Suraj Jayakumar (*Author*)

Abstract— Selecting appropriate health insurance is a complex decision-making task often fraught with information overload. This paper introduces INSURIFIT, a system that enhances personalized health insurance plan recommendations by integrating a robust computational logic layer using Answer Set Programming (ASP). While a conversational interface (Botpress) initially gathers user data—including demographics, location, dependents, tobacco use, and financial preferences—and a Python-based search module filters a large HIOS dataset for candidate plans, the core innovation lies in the subsequent ASP validation and insight generation stage. For each top-ranked plan, user profile attributes and detailed plan features are systematically translated into ASP facts. A dedicated Clingo logic program then applies a declarative set of rules, constraints, and common-sense reasoning. This program infers critical insights not explicitly present in the raw data, such as Medicare eligibility, potential misalignment of user's financial goals; for example desiring both very low premiums and deductibles, the implications of tobacco use, or specific advice like considering family-oriented benefits if dependents are present. These ASP-derived predicates are then fed into a summarization module that utilizes Google's Gemini generative AI. The LLM is prompted to synthesize these logical insights along with direct plan-to-preference comparisons, resulting in enriched, context-aware, and more advisory explanations for the user. This research demonstrates the power of combining data-driven plan searching with declarative ASP reasoning to provide nuanced, explainable, and truly personalized guidance in the complex domain of health insurance selection, moving beyond simple feature matching and mitigating hallucinations.

INTRODUCTION

In today's complex healthcare landscape, selecting an appropriate health insurance plan can be a confusing and time-consuming process. With countless options and variations, consumers often struggle to make informed decisions. To address this challenge, a health insurance chatbot was developed that serves as an intelligent, interactive assistant designed to guide users through the insurance plan selection process.

The chatbot uses structured data about available Texas insurance plans, and, after parsing information provided by the user, finds and summarizes plans that best fit the user's needs.

I. SYSTEM ARCHITECTURE AND DESIGN

A. Overall Structure

The chatbot is split up into six main components. It begins with the User Interface (UI), which provides an intuitive frontend for interaction. The Chatbot (Botpress) manages the conversational flow, adeptly collecting structured user data such as demographics, Texas county of residence, dependent details, tobacco usage, financial preferences (desired premium, deductible, OOP maximums), and specific benefit needs, dynamically constructing a UserProfile object. This profile is then relayed to the backend. The Plan Database, composed of preprocessed Health Insurance Oversight System (HIOS) CSVs, serves as the repository for over 100,000 plan details, internally represented using Python data classes like UserProfile and HIPlanInfo. Following initial data gathering, the Plan Searcher (HISearcher - Python/pandas) receives the UserProfile, performs initial filtering and ranking against the database using explicit criteria, and identifies a set of candidate plans (e.g., top 20 unique plans). The critical computational logic layer is the **ASP Logic/Validation Engine (Python/Clingo)**. For each candidate plan, this engine translates user and detailed plan features into Answer Set Programming (ASP) facts via an asp_utils module. A dedicated Clingo logic program (*insurance_advisor.lp*) then applies declarative rules and common-sense reasoning to infer insights such as Medicare eligibility, potential financial goal misalignments, or advice on family-specific benefits, thereby validating plan suitability beyond simple matching. Finally, the Plan Summarizer (HISummarizer - Python/Google Gemini) receives the UserProfile, the detailed HIPlanInfo for a validated plan, and the list of inferred ASP predicates. It prompts a generative AI model to synthesize these logical insights with direct plan-to-preference comparisons, producing enriched, context-aware, and advisory explanations. Clearly defined interfaces between these components ensure modularity and adaptability throughout the system.

B. Data Sources

The data referenced by the chatbot is derived from a subset of the HIOS 2024 Q1 Data available at cms.gov[3]. This dataset includes numerous files detailing plan attributes, rates, benefits, service areas, and cost-sharing structures. For internal processing, Python data classes (UserProfile, HIPlan, HIPlanInfo) define the structure for user preferences and plan details. A key transformation step involves converting instances

of these classes into logical facts for the ASP Logic/Validation Engine. For example, a user's age becomes `age(currentUser, 47)`. and a plan's premium becomes `plan_premium(plan_id_123, 350)`. This factual representation allows the Clingo solver to apply the declarative rules defined in `insurance_advisor.lp`.

C. Pipeline

The application pipeline begins with user interaction via the UI, where the Chatbot gathers data to form a `UserProfile`. This profile guides the Plan Searcher in querying the Plan Database, identifying an initial list of candidate plans (e.g., top 20 unique HIOS Plan IDs). Subsequently, the ASP Logic/Validation Engine processes each unique candidate plan by converting user and detailed plan data into ASP facts, applying rules from `insurance_advisor.lp` via a Clingo solver to derive inferred predicates representing advice and warnings. These logical insights, along with the `UserProfile` and detailed plan information (as an `HIPlan` object), are then passed to the Plan Summarizer. The summarizer leverages a Gemini LLM to generate comprehensive, conversational explanations that integrate direct plan comparisons with the ASP-derived insights. Finally, these enriched recommendations are relayed back through the Chatbot for presentation to the user.

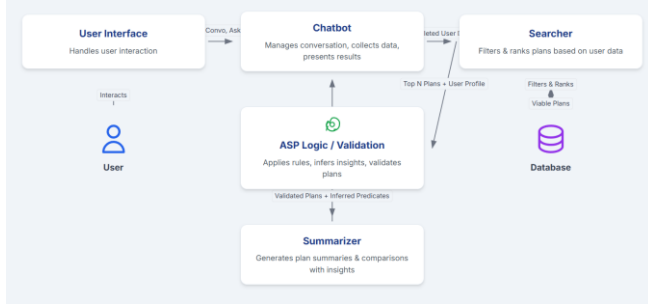


Fig. 1. Chatbot Application Pipeline

D. Packages and Frameworks

The libraries and frameworks used in this application include the following:

- Clingo (CLASP and GRINGO)
- Pandas
- Botpress
- Google genai
- Flask
- ngrok
- Spacy

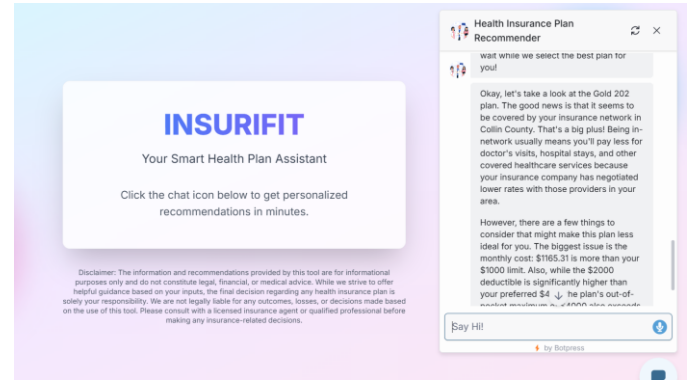


Fig. 2. Chatbot UI

E. Discussions

A. Challenges Faced

1) Crafting precise and comprehensive ASP rules to capture nuanced insurance advice and common-sense reasoning (e.g., Medicare eligibility, financial goal alignment, implications of tobacco use) required significant iteration and **domain understanding**. I obtained this by watching videos by experts on how to pick the right insurance plans.

2) Ensuring the Python `asp_utils.py` module accurately and consistently translated dynamic `UserProfile` and `HIPlanInfo` data into syntactically correct ASP facts for Clingo was crucial and **error-prone, especially regarding atom naming and string literal quoting**. So I carefully normalized all predicate arguments into the correct formats using regex and Python string manipulation.

3) Guiding the Gemini LLM in `HI Summarizer` to effectively incorporate the abstract, logical predicates from ASP into natural, coherent explanations for the user demanded **careful prompt design**. So, I manually wrote explanation strings for some predicates like `advice_consider_family_benefit(User,X)` instead of feeding it to the LLM.

B. Potential Improvements

1) To Expand `insurance_advisor.lp` with more sophisticated rules considering interactions between plan features, user health conditions (if captured), and more granular financial trade-offs. Introduce soft constraints or preferences if supported/simulated.

2) Explore using ASP not just for post-search validation but also more directly in the plan filtering or ranking stages by defining preferences and constraints as part of the ASP query (perhaps using `clpfd`).

3) Investigate methods to provide users with more direct explanations of *how* the ASP engine arrived at certain conclusions, potentially by tracing rule derivations, to further increase transparency and trust and deal with the case where the user asks why the chatbot reasoned the way it did.

C. Lessons Learned

1) ASP (via Clingo) proved highly effective for representing and applying complex, declarative domain

knowledge and business rules, separating logical reasoning from procedural code.

2) Developing ASP rules is an iterative process; starting simple and incrementally adding complexity, with frequent testing against diverse fact sets, is essential.

3) The project successfully demonstrated how ASP's reasoning can complement LLMs pattern matching with ASP providing structured, verifiable insights to guide the LLM and avoid hallucinations.

REFERENCES

- [1] <https://www.texashealthplancompare.com/app/full-benefit>
- [2] <https://www.cms.gov/CCIIO/Resources/Data-Resources/Downloads/HIOS-RBIS-Data-Extract-ICD-020000.pdf>